Week 1 Implementation Foundation

Project: Local Service Problem Diagnosis and Matching System

This report describes the first working version of the project. The goal is to build a Python web application that accepts a household problem, suggests a service category, and saves the request.

1. Source Tree and Main Components

The project uses separate files for the web application, classification code, database work, and web pages. This makes the code easier to read and update.

- **app.py —** Runs the Flask application. It accepts a problem description, calls the classifier, saves the result, and shows the page.
- **classifier.py —** Uses TF-IDF and Logistic Regression to predict a starter category: Plumbing, Electrical, or Appliance Repair.
- **database.py —** Creates the SQLite table and saves or reads service requests.
- **templates/ —** Contains the HTML pages for entering a problem and viewing saved requests.
- **static/style.css —** Controls the basic look of the web pages.
- **requirements.txt —** Lists the Python packages needed to run the application.
- **schema.sql —** Shows the structure of the SQLite request table.

The exact files in the GitHub repository should match the files that are tested. If the current project uses different names, update this list before submitting the report.

2. How to Install and Run the Software

You need Python, pip, and a web browser. In Windows PowerShell, open the project folder and run:

```
cd src
py -m venv .venv
```

.venv\Scripts\Activate.ps1

python -m pip install --upgrade pip

pip install -r requirements.txt

python app.py

After the application starts, open http://127.0.0.1:5000 in your browser. Enter a problem such as “The kitchen sink pipe is leaking” and submit it. Then open the saved-requests page to check whether the request was stored.

The SQLite database is created by the application when it starts. Do not upload the local database file if it contains test or personal information.

3. What Works So Far

In the earlier prototype test, the description “The kitchen sink pipe is leaking” was classified as Plumbing. The page showed a confidence score of 0.51, and the request was saved with ID 1. This test showed that the basic flow from entering a problem to classifying and saving it was working.

The starter code in the supplied bundle may show a different confidence score or request ID. Run the code and record the result you actually see. The training examples are small, so the prediction is only a first estimate and may be wrong.

Provider matching, user accounts, provider accounts, login, and changing request status are not included as completed features in this first version.

4. Changes to the Design or Technology

The project uses Python and Flask for the web application, HTML and CSS for the pages, and SQLite to store requests. The first classifier uses TF-IDF with Logistic Regression because it is simple to run locally. The Week 1 version does not need an external API or cloud service.

The training examples are only for testing the basic flow. They are not large enough to prove that the classifier is accurate. Any changes from the earlier proposal should be added here only after checking the actual code and design.

5. Work Planned for Week 2

- Add and review more example problem descriptions for each service category.
- Prepare test examples that are separate from the training examples.
- Measure accuracy, precision, recall, and F1-score.
- Improve checks for empty or unclear descriptions.
- Write tests for classification and saving or reading requests.
- Define the provider information and matching rules before building provider matching.
- Update the README and this report with real test results.

Week 1 Submission Checklist

☐ Run the application after installing the dependencies.

☐ Submit a test problem and save a screenshot of the result.

☐ Check that the request appears on the saved-requests page or in the database.

☐ Push the source files, requirements, README, and this report to the correct GitHub folders.

Note: This report covers the first implementation foundation. It does not claim that the full system is finished. Make sure the report matches the code and screenshots that you submit.